Facultad de Ingeniería
Escuela de Ingeniería Informática
Cátedra Programación Orientada a Objetos
Profesor Joselito De Sousa

PROYECTO VERSIÓN 1

La isla de las reliquias

Enunciado del proyecto Expedición

Primera fase con Java en consola y API Python

Datos académicos	Información
Asignatura	Programación Orientada a Objetos
Carrera y nivel	Ingeniería Informática de la UCAB, cuarto semestre
Semestre y NRC	202715 - 17032
Fecha de entrega	Semana 9
Equipo	Máximo tres integrantes: defensa grupal 50% con preguntas - defensa individual 50%

Presentación

Los equipos desarrollarán un juego de exploración para dos jugadores humanos. Ambos recorrerán una isla, recogerán reliquias y las entregarán en el campamento. Gana quien acumule más puntos después de ocho rondas. Las ubicaciones y las pistas del cofre cambian al comenzar una partida nueva.

Esta versión concentra las reglas en movimiento, inventario, entrega de reliquias y un cofre cifrado. Cada jugador dispone de tres puntos de acción por turno y las mismas capacidades. Para esta fase el tablero tiene 25 casillas pero esto no quiere decir que se mantiene para la segunda y cada partida contiene cinco reliquias: cuatro normales y una especial.

Alcance académico

Se entregará una aplicación Java de consola con registro, juego, carga de partidas y estadísticas en un mismo menú. La API Python genera los escenarios y conserva los datos.

La segunda fase incorporará JavaFX reutilizando el modelo. En esta entrega se evalúan POO, estructuras de datos, MVC, aplicación de dos patrones (NO SINGLETON), consumo de API y descifrado mediante bibliotecas estándar.

Cambios y alcance de la versión reducida

La reducción de complejidad es aproximada y se apoya en retirar ocho grupos de mecánicas: habilidades de personajes, eventos, herramientas, trampas, observación, descarte, verificación de documentos y bonificaciones. También se acortan el tablero y la duración. Se conserva la exploración competitiva y el uso de criptografía real en el cofre.

Aspecto	Versión anterior	Versión reducida
Tablero	7 × 7, 49 casillas	5 × 5, 25 casillas
Participantes	Dos a cuatro	Exactamente dos
Duración	Doce rondas	Ocho rondas
Acciones por turno	Tres o cuatro según evento	Siempre tres
Terrenos	Siete tipos	Cinco tipos
Personajes y habilidades	Cuatro roles distintos	Capacidades iguales
Eventos y bonificaciones	Mazo y bonos por ronda	Eliminados
Herramientas y trampas	Puentes, escudos y dos trampas	Eliminados
Observación y descarte	Acciones y estados propios	Eliminados
Templo normal	Explorar y recoger por separado	Explorar y recoger en una acción
Pistas del cofre	Miradores, firmas, falsos y horas	Dos fragmentos numerados en templos
Apertura del cofre	Descifrar, código y recogida separados	Descifrar y recoger en una acción
Aplicaciones Java	Juego y gestor independientes	Una aplicación con un menú principal
Guardado	Después de cada acción	Al cerrar el turno, manualmente y al salir

Requisitos que se mantienen

Ubicaciones aleatorias, movimiento ortogonal, costo adicional de pantanos, capacidad de inventario, puntuación por entrega, ganador y desempate, persistencia JSON, API Python, MVC y preparación para JavaFX. El cofre continúa usando PBKDF2 y AES-GCM, con datos nuevos por partida.

Los únicos objetos transportables son reliquias. No hay objetos en el suelo, intercambios, combate ni habilidades especiales. Los ejemplos de posiciones pertenecen a una configuración de prueba y no fijan el mapa de todas las partidas.

1 Tablero y ubicación de los elementos

La matriz tiene cinco filas y cinco columnas (pero no debe ser fijo el tamaño ya que puede variar para la segunda versión). Las columnas se nombran A–E y las filas 1–5; A1 es la esquina superior izquierda. La API utiliza índices fila y columna de 0 a 4. Todas las casillas se pueden atravesar y dos fichas pueden compartir la misma posición.

Terreno	Símbolo	Cantidad	Uso
Campamento	C	1	Inicio compartido y entrega de reliquias
Sendero	S	15	Movimiento normal
Pantano	P	4	Entrada de costo dos
Templo normal	T	4	Una reliquia, y en dos de ellos una pista
Templo del cofre	X	1	Reliquia especial protegida por AES

Expedición Tablero de ejemplo EX-RED-01

	A	B	C	D	E
1	A1 **S** Sendero	B1 **T** Templo	C1 **S** Sendero	D1 **P** Pantano	E1 **S** Sendero
2	A2 **P** Pantano	B2 **S** Sendero	C2 **S** Sendero	D2 **X** Cofre	E2 **S** Sendero
3	A3 **S** Sendero	B3 **T** Templo	C3 **C** Campamento	D3 **S** Sendero	E3 **P** Pantano
4	A4 **S** Sendero	B4 **S** Sendero	C4 **T** Templo	D4 **S** Sendero	E4 **S** Sendero
5	A5 **T** Templo	B5 **P** Pantano	C5 **S** Sendero	D5 **S** Sendero	E5 **S** Sendero

Campamento
Sendero
Pantano
Templo
Cofre

Inicio compartido C3

Cofre D2

Las ubicaciones se sortean al crear una partida y se conservan al cargarla.

El mapa y los tipos de casilla son visibles desde el inicio. El contenido de cada templo permanece oculto hasta explorarlo. Los dos puntos de color identifican las fichas de Ana y Luis (nombre al azar) en el campamento C3.

2 Turnos y acciones del jugador

La API sortea el orden inicial de los dos participantes y lo conserva. Un turno es la intervención de un jugador; una ronda contiene un turno de cada uno. Cada turno comienza con tres puntos de acción. Los puntos no usados se pierden al terminarlo.

Acción	Costo	Condición
Mover a una casilla que no es pantano	1	Destino vecino ortogonal y dentro del tablero
Entrar en un pantano	2	Debe haber al menos dos puntos disponibles
Explorar y recoger una reliquia	1	Estar en T, no explorado, con espacio en inventario
Entregar todas las reliquias	1	Estar en C y llevar al menos una
Intentar abrir el cofre	1	Estar en X, dos pistas conocidas, cofre cerrado y espacio
Consultar tablero, inventario o pistas	0	Solo muestra información ya disponible
Guardar, leer ayuda o terminar turno	0	No concede puntos ni regenera datos. EL guardado es Automático por cada movimiento.

Validación de las acciones

El movimiento permite arriba, abajo, izquierda o derecha, sin diagonales ni saltos. Su costo depende del destino. Entrar en P cuesta dos; salir de P hacia S cuesta uno. No se puede ejecutar una acción si excede el presupuesto.

Una opción inexistente, coordenada fuera del mapa, movimiento inválido, inventario lleno o acción incompatible no altera el estado ni consume puntos. Un intento de contraseña con formato admisible sí consume un punto cuando el descifrado falle.

Cierre del turno y de la partida

Al llegar a cero se cierra el turno automáticamente. También puede cerrarse antes con Terminar turno. Java prepara el turno siguiente y guarda el estado antes de permitir nuevas acciones. Después del segundo jugador aumenta la ronda. Al finalizar la ronda ocho se registra FINALIZADA, sin conceder acciones adicionales.

El límite se expresa en rondas. No hay cuenta regresiva en segundos ni eventos que cambien costos. Si un turno comienza con tres puntos, se pueden mover dos casillas y explorar; o entrar en un pantano y realizar otra acción de costo uno. Es importante guardar el tiempo consumido en segundos en cada turno por cada jugador.

3 Reliquias inventario y puntuación

Templos normales

Cada uno de los cuatro templos contiene una reliquia. Sus valores son 1, 2, 2 y 3 puntos, asignados al azar. Dos templos distintos contienen además una pista numerada. Explorar revela el contenido y transfiere la reliquia al inventario en la misma acción.

Si se encuentra una pista, queda registrada inmediatamente y ambos jugadores pueden consultarla. Explorar un templo ya explorado solo muestra su estado y no cobra una acción. Su reliquia no reaparece. Si el inventario está lleno, se rechaza la exploración sin revelar la pista ni retirar el objeto.

Inventario

Cada jugador puede llevar como máximo dos reliquias. Las pistas están en un registro compartido y no ocupan espacio. No existe descarte: para liberar espacios hay que regresar al campamento y entregar. Nadie puede tomar objetos del inventario de otro participante.

Entrega y ganador

Entregar cuesta una acción y retira todas las reliquias del inventario, sumando sus valores. No permite elegir solo algunas. Los objetos se registran como entregados y dejan de estar disponibles. No se reciben puntos por descubrir, transportar o simplemente estar en el campamento.

La reliquia especial vale cuatro puntos y se obtiene al abrir el cofre. El total de puntos disponibles es doce: ocho de las reliquias normales y cuatro de la especial. No hay bonificaciones. Abrir el cofre no termina la expedición.

Al finalizar la ronda ocho gana quien tenga más puntos. En empate se compara el número de reliquias entregadas. Si también coincide, ambos comparten la victoria. Las reliquias que queden en inventario no puntúan.

Ejemplo

Inventario: REL-01 de 3 puntos y REL-02 de 2 puntos
Posición: campamento | Acciones: 2 | Puntos actuales: 0
Entregar: costo 1, inventario vacío, puntuación 5.
Acciones restantes: 1.

Si el jugador llega a C con cero acciones y termina la última ronda, sus objetos quedan sin entregar. El programa conserva el inventario final para mostrar el resultado.

4 Generación aleatoria y responsabilidades

Datos que genera Python

La API genera posiciones de C, X, cuatro T y cuatro P; las demás son S. Distribuye los valores de las reliquias, elige los dos templos con pistas, produce sus fragmentos y el paquete cifrado, y sortea el orden de los jugadores. Todos los participantes empiezan en el campamento generado.

1. Conservar las 25 casillas y cantidades indicadas, sin superposición entre terrenos.

2. Dar al campamento al menos dos vecinos que no sean pantanos.

3. Situar al menos un templo normal a dos movimientos o menos de C mediante una ruta sin pantanos.

4. Garantizar una ruta sin pantanos C → templo de pista 1 → templo de pista 2 → C → X → C de hasta doce movimientos, permitiendo intercambiar el orden de los dos templos.

5. Asignar una pista de índice 1 y una de índice 2 a dos templos diferentes, y comprobar que el archivo generado se descifra con su contraseña.

La visita intermedia a C libera el inventario después de recoger dos reliquias. Doce movimientos, dos exploraciones, dos entregas y una apertura requieren diecisiete puntos de acción, dentro de los veinticuatro disponibles en ocho turnos de un jugador. Esto garantiza la viabilidad del objetivo; los rivales pueden disputar las reliquias.

Para un mismo par de jugadores, las coordenadas de C y X deben cambiar respecto de la partida anterior. Los fragmentos también cambian. Las otras posiciones se sortean y pueden coincidir ocasionalmente. Se admite hasta mil candidatos antes de devolver un error de generación, sin crear una partida incompleta.

4.1 Responsabilidades Técnicas del Backend en Python

El estudiante deberá programar la lógica del servidor cumpliendo estrictamente con el **Contrato de la API v2** (Sección 6 del documento):

1. **Generación Aleatoria del Escenario y Validación de Rutas (Sección 4):**
    - **Generar la matriz 5×5 respetando la cantidad exacta de casillas (1C, 1X, 4T, 4P, 15S).**

      1. Las letras con números (1 C, 1 X, 4 T, 4 P, 15 S)
      Las letras representan los tipos de casilla en el tablero de 5 por 5 (25 casillas en total):

      - **C**: Campamento (inicio y punto para entregar reliquias).
      - **X**: Templo del cofre (casilla donde se encuentra la reliquia especial).
      - **T**: Templo normal (casillas donde hay reliquias y pistas).
      - **P**: Pantano (terreno que cuesta 2 puntos atravesar).
      - **S**: Sendero (terreno de movimiento normal que cuesta 1 punto).

El número indica la cantidad exacta de casillas de ese tipo que deben existir en la partida (1 + 1 + 4 + 4 + 15 = 25 casillas).

- **Validar las restricciones geométricas (p. ej., Campamento con ≥ 2 vecinos no pantanos, rutas sin pantanos viables en ≤ 12 movimientos, etc.).**

2. Los símbolos de comparación (mayor o igual a 2, menor o igual a 12)

Representan reglas de validación que debe cumplir la API en Python al momento de generar un mapa válido:

- **Mayor o igual a 2 vecinos no pantanos (>= 2):** El Campamento (C) debe tener por lo menos 2 casillas contiguas (arriba, abajo, izquierda o derecha) que **no** sean Pantanos (P).
- **Menor o igual a 12 movimientos (<= 12):** Existe un camino sin pantanos que conecta el Campamento con los templos de las pistas, el cofre y de regreso al Campamento que toma como máximo 12 pasos.

2. **Cifrado del Cofre (Sección 5 y Anexo B):**
   - Implementar en Python la generación del paquete cifrado utilizando cryptography o PyCryptodome:
     - Derivación de clave mediante **PBKDF2WithHmacSHA256** (100,000 iteraciones, clave de 256 bits, salt de 16 bytes).
     - Cifrado autenticado **AES-GCM** (Nonce de 12 bytes, Tag de 128 bits, AAD = "partidaId|cofreId").
     - Codificación Base64 para el transporte de los parámetros.
3. Persistencia y Control de Concurrencia/Revisiones (Secciones 7 y 8):
   - Almacenar datos en formato JSON mediante escritura atómica (archivos temporales e intercambio).
   - Implementar la gestión de revisiones (revisionBase) para evitar sobreescritura conflictiva y garantizar idempotencia en peticiones duplicadas.
   - Cálculo de estadísticas consolidadas a partir de partidas finalizadas únicas.

4.1.1. Requisitos de Documentación: Swagger y Postman

A. Documentación interactiva en Swagger (OpenAPI)

- La API debe exponer la interfaz gráfica interactiva de Swagger UI en la ruta /docs (o /redoc).

- Cada endpoint expuesto en la API debe contar con:
    - Descripción clara de la operación.
    - Códigos de estado HTTP adecuados (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 409 Conflict, 503 Service Unavailable).
    - Esquema de entrada (Payload JSON) y salida (Response Body) correctamente tipados.

B. Colección de Postman (Entregable obligatorio)

El estudiante deberá incluir un archivo de exportación de Postman (.json) con:

1. Colección con todos los endpoints del Contrato v2:
    - GET /salud
    - POST /jugadores y GET /jugadores
    - GET /jugadores/{id}/estadisticas
    - POST /partidas y GET /partidas
    - GET /partidas/{id}
    - GET /partidas/{id}/templos/{coord}
    - GET /partidas/{id}/cofre
    - PUT /partidas/{id}/estado

Endpoint	Descripción
GET /salud	Comprueba la disponibilidad del servicio y la versión del contrato de la API.
POST /jugadores	Registra un nuevo jugador enviando su nombre y alias, devolviendo su identificador (jugadorId).
GET /jugadores	Consulta y lista los registros de jugadores, permitiendo un filtro opcional por alias.
GET /jugadores/{id}/estadisticas	Obtiene las estadísticas de un jugador (partidas finalizadas, victorias y puntos acumulados).
POST /partidas	Genera y crea una nueva partida (incluyendo la distribución del tablero, cofre y pistas) y retorna la configuración pública y estado inicial.
GET /partidas	Consulta y lista resúmenes de partidas, con posibilidad de filtrar por jugadorId.
GET /partidas/{id}	Recupera la configuración inmutable y el estado guardado de una partida específica.
GET /partidas/{id}/templos/{coord}	Obtiene la información del contenido de un templo en una coordenada dada (su reliquia y pista opcional).
GET /partidas/{id}/cofre	Devuelve el paquete cifrado completo del cofre para la partida especificada.
PUT /partidas/{id}/estado	Guarda y actualiza el estado completo o *snapshot* de una partida mediante control de revisión.

2. **Ejemplos guardados (Examples/Responses):** Incluir casos de prueba exitosos (200/201) y casos de error con sus códigos correspondientes (400, 409, etc.).
3. **Variables de entorno de Postman:** Configurar una variable {{base_url}} definida por defecto como http://localhost:8000/api/v1.

4.1.2. Guía para el Despliegue en Local

El estudiante debe incluir en la entrega (en el README.md del proyecto) las instrucciones completas para levantar la API localmente:

1. **Entorno de ejecución:** Indicación de la versión de Python (Python 3.10+ recomendado).
2. **Gestión de dependencias:** Incluir un archivo requirements.txt con librerías como:

   fastapi>=0.100.0
   uvicorn>=0.20.0
   pydantic>=2.0
   cryptography>=41.0.0
   Comando de inicio del servidor:

   uvicorn main:app --reload --port 8000

   Verificación de la salud del sistema: Probar en el navegador o consola:

   GET http://localhost:8000/api/v1/salud
   Debe responder: {"disponible": true, "versionContrato": 2}.

Para desarrollar y desplegar la API con FastAPI localmente para el proyecto de la expedición, necesitas los siguientes componentes y seguir unos pasos sencillos de configuración:

1. ¿Qué necesitas? (Requisitos)

- Python 3.10 o superior instalado en tu sistema.
- Entorno virtual de Python (venv): Recomendado para aislar las librerías del proyecto.
- Paquetes principales de Python:
  - fastapi: El framework para definir las rutas y contratos HTTP.
  - uvicorn[standard]: El servidor ASGI ligero para ejecutar FastAPI en local.
  - pydantic: Para la validación de datos y schemas JSON automáticamente.
  - cryptography o pycryptodome: Para implementar el cifrado PBKDF2 y AES-GCM del cofre.

2. Paso a paso para desplegar en local

Paso 1: Crear la estructura del proyecto y el entorno virtual
Abre tu terminal en la carpeta de tu proyecto backend y ejecuta:

```
# Crear el entorno virtual
python -m venv venv
```

```
# Activar el entorno virtual
# En Windows (PowerShell):
.\venv\Scripts\Activate.ps1
# En Linux / macOS:
source venv/bin/activate
```

Paso 2: Crear el archivo requirements.txt e instalar dependencias

Crea un archivo llamado requirements.txt con el siguiente contenido:

```
fastapi>=0.100.0
uvicorn[standard]>=0.20.0
pydantic>=2.0.0
cryptography>=41.0.0
```

E instala los paquetes ejecutando:

```
pip install -r requirements.txt
```

Paso 3: Crear el código base (main.py)

Crea un archivo main.py estructurando la URL base /api/v2 requerida por el contrato del juego:

```
from fastapi import FastAPI
from pydantic import BaseModel
# Inicializar la app con el prefijo /api/v2
app = FastAPI(
   title="API Expedición Isla Reliquias",
   version="2.0.0",
   docs_url="/docs",  # Documentación Swagger automática
   redoc_url="/redoc"
)
# Endpoint de salud según el contrato
@app.get("/api/v2/salud")
def consultar_salud():
   return {
      "disponible": True,
      "versionContrato": 2
   }
```

Paso 4: Levantar el servidor Uvicorn

Para iniciar el servidor local con recarga automática al hacer cambios en el código (--reload), ejecuta en la terminal:

```
uvicorn main:app --reload --port 8000
```

3. Comprobación y Verificación

1. **Prueba de servicio activo:**
   Abre tu navegador e ingresa a:
   http://localhost:8000/api/v1/salud
   Debe responder: {"disponible": true, "versionContrato": 2}.

2. **Swagger UI automático:** Ingresa a: http://localhost:8000/docs Allí verás la interfaz interactiva de Swagger generada automáticamente, donde podrás probar todos los endpoints (POST /jugadores, POST /partidas, GET /partidas/{id}/cofre, etc.).

4.2. Responsabilidad de Java

Java aplica movimiento, presupuesto (puntos de acción), inventario, exploración, registro de pistas, descifrado, entregas, turnos y victoria. La vista lee e imprime; el controlador consulta la API cuando necesita contenido y coordina las acciones del modelo. La API conserva configuraciones y snapshots, es decir, copias completas del estado.

La configuración de una partida no vuelve a sortearse al guardar o cargar. La semilla del mapa, si se usa para diagnóstico, se conserva. Sal y nonce utilizan aleatoriedad apropiada independiente de la semilla del mapa.

5 Cofre cifrado con dos pistas

Dos templos normales guardan fragmentos que se revelan al explorarlos. La pista 1 es una palabra de seis letras A–Z y la pista 2 una cadena de cuatro dígitos. Se unen en ese orden mediante un guion: por ejemplo, ORBITA-7392. Se conservan ceros iniciales. No hay firmas, pistas falsas ni ordenamiento por horas son esos dos fragmentos en esta versión.

El jugador debe escribir la contraseña en X. La pantalla muestra los fragmentos conocidos, pero no los concatena. Se admiten de cuatro a veinte caracteres con el patrón [A-Z0-9]+(?:-[A-Z0-9]+)?; las minúsculas se convierten a mayúsculas, sin quitar espacios. ORBITA7392 tiene formato admisible y produce un intento fallido.

¿Cómo actúa la interfaz y el programa?

- **No la regala:** La pantalla del juego le mostrará al jugador los fragmentos que ha descubierto (ej. Pista 1: ORBITA y Pista 2: 7392), pero **no mostrará la contraseña ya unida**. El jugador debe concatenarla y escribirla él mismo.
- **Formato aceptado (expresión regular [A-Z0-9]+(?:-[A-Z0-9]+)?):**
    - El sistema permite ingresar textos que tengan de **4 a 20 caracteres** con letras y números, con o sin guion.
    - Si el jugador escribe en minúsculas (ej. orbita-7392), el programa automáticamente las pasa a **mayúsculas** (ORBITA-7392).
    - Los espacios **no** se eliminan automáticamente.

Diferencia entre "Formato Admisible" y "Contraseña Correcta"

Este es el punto clave que ejemplifica el texto:

1. **Intento fallido que consume acción (ORBITA7392):**
    - Si el jugador olvida colocar el guion e ingresa ORBITA7392, la cadena cumple con el formato permitido (contiene letras y números de 4 a 20 caracteres).
    - Como el formato es válido, el juego intenta descifrar el cofre con esa clave.
    - El descifrado criptográfico **falla**, por lo que el cofre no se abre y **se descuenta 1 punto de acción** del turno del jugador.
2. **Error de formato (no consume acción):**
    - Si ingresa algo fuera de norma (como caracteres especiales prohibidos o menos de 4 letras), el sistema lo rechaza por formato inválido antes de intentar descifrar, por lo que **no cobra puntos de acción**.

Donde se encuentran las pistas:

Ubicación exacta: Hay un total de 4 templos normales en el tablero (identificados con la letra **T**). De esos 4 templos, **exactamente dos contienen una pista** (elegidos al azar al generar la partida).
Cómo obtenerlas: Las pistas se revelan cuando un jugador realiza la acción de **explorar** un templo normal que contenga una.
Registro de pistas: Al descubrir una pista, esta se registra inmediatamente en un registro compartido y queda visible para ambos jugadores.

Parámetro	Contrato
Derivación de clave	PBKDF2WithHmacSHA256, 100000 iteraciones, salida 256 bits
Sal	16 bytes aleatorios
Cifrado	AES/GCM/NoPadding, clave de 32 bytes
Nonce	12 bytes, sin repetir para la misma clave
Etiqueta	128 bits, incluidos como últimos 16 bytes del ciphertext
AAD	UTF-8 de partidaId + "|" + cofreId
Contenido antes de cifrar	JSON UTF-8 con partidaId, cofreId, reliquiaId y valor=4
Transporte	Sal, nonce, AAD y ciphertext con etiqueta en Base64

Apertura en una sola acción

Java valida posición, dos pistas conocidas, presupuesto (puntos de acción) y espacio; obtiene el paquete, deriva la clave y descifra con la biblioteca estándar. Si autentica el archivo y sus identificadores coinciden, marca el cofre abierto y agrega REL-ESPECIAL al inventario. Toda la operación cuesta un punto. La entrega en C sigue siendo otra acción.

Si la autenticación falla, cobra un punto y muestra No se pudo autenticar el archivo. Si el paquete tiene Base64 roto, tamaños incorrectos, campos ausentes o identificadores incompatibles, informa un error de datos sin cobrar. Un cofre abierto muestra su estado sin costo y nunca produce una segunda reliquia.

La API no entrega contraseña, clave AES ni texto descifrado como campos públicos. Java usa SecretKeyFactory, PBEKeySpec, Cipher, SecretKeySpec y GCMParameterSpec. No se implementan las primitivas desde cero. Las referencias técnicas y un vector completo están al final del documento.

6 Contrato de la API Python

URL base configurable: http://localhost:8000/api/v1. Los cuerpos y respuestas usan JSON UTF-8. La versión 2 sustituye el contrato del enunciado anterior para este alcance. GET consulta, POST crea y PUT guarda el estado completo.

Método y ruta	Entrada	Respuesta principal
GET /salud	Sin cuerpo	Disponible y versionContrato=2
POST /jugadores	nombre, alias	Jugador creado con jugadorId
GET /jugadores	Filtro opcional alias	Lista de registros
GET /jugadores/{id}/estadisticas	Sin cuerpo	Partidas, victorias y puntos
POST /partidas	requestId, jugadores	Configuración pública y estado inicial
GET /partidas	Filtro jugadorId	Resúmenes de partidas
GET /partidas/{id}	Sin cuerpo	Configuración y estado guardado
GET /partidas/{id}/templos/{coord}	Sin cuerpo	Reliquia y pista opcional
GET /partidas/{id}/cofre	Sin cuerpo	Paquete cifrado completo
PUT /partidas/{id}/estado	revisionBase, estado	Revisión y guardado confirmado

Contenido y validaciones

Las consultas de templo y cofre no gastan acciones ni alteran la partida. Java comprueba la posición y aplica el efecto. La API verifica el tipo de casilla y los identificadores; no compara la posición con un guardado antiguo, porque el juego guarda principalmente al terminar el turno.

La creación devuelve solo tipos de terreno, posiciones, participantes, orden y estado inicial. Los contenidos se recuperan al explorar; los datos ya obtenidos permanecen en memoria y en el snapshot. No se solicita proteger esta actividad académica contra clientes modificados.

Respuestas de error

HTTP 400 indica datos inválidos; 404, recurso inexistente; 409, alias repetido, requestId conflictivo o revisión incompatible; 503, servicio no disponible o generación fallida. Los errores usan {"codigo":"...","mensaje":"..."}. Java valida respuesta, campos y tipos antes de aplicar el cambio.

El primer guardado FINALIZADA fija el resultado. El backend calcula estadísticas sobre partidas finalizadas únicas, sin sumar otra vez al repetir un guardado. No se necesita un endpoint adicional de resultados, eventos ni apertura por código.

7 Datos y guardado de una partida

Para el registro del jugador en el juego se debe solicitar:

- El nombre tiene de uno a ochenta caracteres.
- El alias tiene de tres a veinte, con letras, dígitos, guión o guion bajo, y es único sin distinguir mayúsculas.
- No se requieren correo ni contraseña. Una partida utiliza dos jugadores registrados y distintos.

Aplicar validaciones con expresiones regulares a estos valores.

```
POST /partidas
{"requestId":"solicitud-001","jugadores":["J1","J2"]}

Ejemplo de PUT /partidas/EX-RED-01/estado
{
 "revisionBase":2,
 "estado":{
  "estado":"EN_CURSO","ronda":2,
  "indiceJugadorActual":0,"accionesRestantes":3,
  "jugadores":[
   {"jugadorId":"J1","posicion":"C3","puntos":0,
    "inventario":[{"reliquiaId":"REL-01","valor":3}],
    "entregadas":[]},
   {"jugadorId":"J2","posicion":"D4","puntos":0,
    "inventario":[{"reliquiaId":"REL-02","valor":2}],
    "entregadas":[]}
  ],
  "templosExplorados":["B3","C4"],
  "pistas":[{"indice":1,"fragmento":"ORBITA"},
        {"indice":2,"fragmento":"7392"}],
  "cofreAbierto":false,"ganadores":[]
 }
}
```

El estado del ejemplo corresponde al comienzo de la ronda dos, después de los dos turnos de la primera. La configuración inmutable se almacena por separado: mapa, valores y pistas asignados, orden de jugadores y paquete cifrado. Cada reliquia conserva su id y valor al pasar de templo a inventario y a entregadas.

La creación con un requestId ya usado y el mismo cuerpo devuelve la partida existente; si cambia el cuerpo, devuelve 409. El cliente conserva ese id hasta recibir confirmación. El snapshot incluye las pistas compartidas y las acciones pendientes; cargarlo no reinicia el turno.

EJemplo:

Registro del jugador (POST /jugadores)
Al registrar un usuario con su nombre y alias, el backend (Python) genera un jugadorId único para él (por ejemplo, "J1" para Ana y "J2" para Luis):

POST /jugadores
{ "nombre": "Ana Pérez", "alias": "ana_ucab" }

// Respuesta del servidor: { "jugadorId": "J1", ... }

Creación de la partida (POST /partidas)

```
POST /partidas
{
  "requestId": "solicitud-001",
  "jugadores": ["J1", "J2"]
}
```

- **Propósito:** Le solicita al servidor (API Python) que genere una nueva partida para dos jugadores (J1 y J2).
- **requestId ("solicitud-001"):** Es un identificador único generado por Java para garantizar la **idempotencia**. Si el mensaje se reenvía por un fallo de red, la API no creará otra partida duplicada, sino que devolverá la misma partida creada anteriormente.
- **jugadores:** Lista con los identificadores de los dos jugadores previamente registrados.

2. Petición para guardar/actualizar el estado (PUT /partidas/{id}/estado)

```
PUT /partidas/EX-RED-01/estado
{
  "revisionBase": 2,
  "estado": {
    "estado": "EN_CURSO",
    "ronda": 2,
    "indiceJugadorActual": 0,
    "accionesRestantes": 3,
    "jugadores": [
      {
        "jugadorId": "J1",
        "posicion": "C3",
        "puntos": 0,
        "inventario": [{"reliquiaId": "REL-01", "valor": 3}],
        "entregadas": []
      },
      {
        "jugadorId": "J2",
        "posicion": "D4",
        "puntos": 0,
        "inventario": [{"reliquiaId": "REL-02", "valor": 2}],
        "entregadas": []
      }
    ],
    "templosExplorados": ["B3", "C4"],
    "pistas": [
      {"indice": 1, "fragmento": "ORBITA"},
```

```
    {"indice": 2, "fragmento": "7392"}
   ],
   "cofreAbierto": false,
   "ganadores": []
  }
}
```


Explicación del contenido del estado:

- **estado ("EN_CURSO"):** Indica que la partida está activa (cambiará a "FINALIZADA" al terminar la ronda 8).
- **ronda (2) / accionesRestantes (3) / indiceJugadorActual (0):** La partida va iniciando la ronda 2. Le toca al jugador 0 (J1), quien comienza con sus 3 puntos de acción.
- **jugadores:** Muestra la ubicación actual de cada participante, los puntos sumados por entregas, lo que llevan cargado en inventario y lo que ya han entregado.
- **templosExplorados (["B3", "C4"]):** Lista de casillas de templo que ya fueron descubiertas y vaciadas.
- **pistas:** Registro compartido con los fragmentos descubiertos hasta el momento (ORBITA y 7392).
- **cofreAbierto (false) / ganadores ([]):** Indican que la reliquia especial sigue en el cofre y aún no hay un ganador definido.

- **Propósito:** Guarda un **snapshot** (fotografía completa) del estado actual de la partida EX-RED-01 en el servidor.
- **revisionBase:2:** Mecanismo de control de concurrencia. Informa a la API en qué versión se basó este cambio para evitar sobrescribir estados desactualizados o duplicar guardados.

  El campo **revisionBase** en la petición PUT /partidas/{id}/estado es la pieza central del **control de concurrencia optimista** del juego. Su función principal es **evitar la pérdida de datos por sobrescritura y prevenir la duplicación no deseada de estado o estadísticas**.

  1. ¿Qué problema resuelve?

  En sistemas cliente-servidor (como la aplicación Java interactuando con la API Python), pueden ocurrir dos problemas comunes al guardar información:

  - **Sobrescritura accidental (Race conditions):** Si dos clientes intentan modificar la misma partida al mismo tiempo partiendo de un mismo punto, el último en guardar borraría las acciones del primero.
  - **Reintentos de red (Duplicación):** Si la aplicación Java envía un guardado pero la conexión falla al recibir la confirmación, el cliente volverá a intentar enviar la petición. Sin un control de versión, la API no sabría si se trata de la misma jugada reintentada o de un movimiento nuevo.

  2. ¿Cómo funciona en el flujo del juego?

  1. **Lectura inicial:** Cuando Java consulta el estado actual de la partida, la API devuelve el snapshot con su número de revisión guardado (por ejemplo, revisionBase = 2).
  2. **Ejecución del turno:** Los jugadores realizan sus acciones en la aplicación Java.

3. **Envío del guardado:** Al cerrar el turno o guardar manualmente, Java envía a la API el nuevo estado incluyendo "revisionBase": 2.
4. **Validación en la API:**
    - La API verifica si la versión guardada en su base de datos coincide con el revisionBase recibido.
    - Si **coincide**, la API guarda el snapshot y **aumenta la versión a 3** (devuelve HTTP 200/201).

3. Casos de respuesta de la API segun revisionBase

- **Guardado exitoso:** La revisión actual en la API coincide con revisionBase. La API actualiza los datos y retorna el nuevo número de revisión (incrementado en 1).
- **Reintento seguro (Idempotencia):** Si Java reintenta enviar **exactamente el mismo snapshot** con el mismo revisionBase (por ejemplo, tras perder una conexión de red), la API reconoce que esa jugada ya se procesó, no duplica los datos ni las estadísticas y responde con la misma confirmación previa.
- **Conflicto de versión (HTTP 409 Conflict):** Si Java envía revisionBase: 2, pero la API ya tiene guardada la versión 3 (porque otro proceso o cliente actualizó la partida antes), la API rechaza el cambio para evitar que un estado desactualizado destruya la información más reciente.

8 Persistencia y recuperación sencilla

Java guarda automáticamente al cerrar cada turno, y también al seleccionar Guardar o Guardar y salir. La API conserva JSON mediante escritura temporal y reemplazo del archivo definitivo. Solo se abre una partida por proceso y no se exige edición simultánea desde varios clientes.

Revisiones y reintentos

Cada snapshot se envía con revisionBase. Si coincide con la revisión actual, se guarda y la API devuelve la revisión siguiente. Repetir exactamente el mismo snapshot con su base anterior devuelve la misma confirmación, sin duplicar efectos. Reutilizar esa base con otro estado devuelve 409. El cliente no recalcula ni ejecuta acciones para reintentar el guardado.

Si se pierde la respuesta, Java consulta GET /partidas/{id}: cuando la revisión y el estado coinciden con el snapshot pendiente, acepta la confirmación; si no, mantiene el pendiente y muestra el conflicto. No se exige actionId por cada operación.

Fallos de comunicación

Si falla una consulta necesaria antes de ejecutar una acción, no se cobra ni cambia el estado. Si falla el guardado, se conserva en memoria el snapshot pendiente y se bloquean nuevas acciones de juego hasta reintentarlo. Se pueden consultar datos ya disponibles. Guardar y salir solo cierra normalmente tras una confirmación.

Esta versión no exige un archivo de respaldo local ni una pantalla para resolver versiones divergentes. La recuperación garantizada comienza en el último guardado confirmado. Las acciones posteriores a ese guardado pueden perderse si se cierra el proceso de forma abrupta.

Estado final y estadísticas

Al terminar se guarda FINALIZADA, ronda=8, accionesRestantes=0, los ganadores y los inventarios finales. indiceJugadorActual conserva el índice del último jugador; en una partida finalizada no representa un turno pendiente. Se permite consultar el resultado, pero no continuar jugando.

Las estadísticas mínimas son:

- partidas finalizadas, victorias, puntos acumulados y reliquias entregadas.
- Un empate compartido cuenta como victoria para ambos.
- El backend las obtiene de los resultados finales únicos.
- Se eliminan duración, ranking global y contadores de aperturas y movimientos como requisitos obligatorios.

9 Menús y registro de jugadores

El menú principal reúne el registro, las partidas y las estadísticas. No se entrega un gestor independiente. Se deben conservar mensajes comprensibles, validación de números y retorno al menú después de cada operación.

```
EXPEDICIÓN  LA ISLA DE LAS RELIQUIAS
1. Registrar jugador
2. Nueva partida
3. Cargar partida
4. Consultar estadísticas
5. Reglas y ayuda
0. Salir

Registro
Nombre: Ana Pérez
Alias: ana_ucab
Jugador creado: J1

Nueva partida
Jugador 1: ana_ucab
Jugador 2: luis_ucab
Partida: EX-RED-01 | Campamento: C3
Orden sorteado: Ana, Luis | Ronda: 1 de 8
```

Carga y consultas

Cargar muestra identificador, jugadores, estado y ronda de las partidas del jugador seleccionado. Las partidas EN_CURSO recuperan exactamente el snapshot; las partidas FINALIZADAS abren su resultado. No se permite repetir el mismo jugador como los dos participantes.

```
ESTADÍSTICAS DE ANA
Partidas finalizadas: 1
Victorias: 1
Puntos acumulados: 8
Reliquias entregadas: 3

ESTADÍSTICAS DE LUIS
Partidas finalizadas: 1
Victorias: 0
Puntos acumulados: 4
Reliquias entregadas: 2
```

Los datos estadísticos anteriores suponen que la partida de ejemplo es la primera finalizada por ambos. No se solicita eliminación o edición de registros ni autenticación avanzada.

10 Pantallas del turno y del cofre

```
EX-RED-01 | Ronda 2 de 8
Ana | Posición: C3 | Puntos: 0 | Acciones: 3
Inventario: REL-01, valor 3

1. Mover
2. Explorar templo
3. Entregar reliquias
4. Abrir cofre
5. Consultar tablero, inventario y pistas
6. Guardar
7. Terminar turno
0. Guardar y salir

Destino: D4
Movimiento inválido: no es vecino ortogonal de C3.
Acciones restantes: 3.
```

Pistas y apertura

```
PISTAS COMPARTIDAS
Pista 1: ORBITA | Pista 2: 7392
Regla: unir pista 1, guion y pista 2.

Ana en D2 | Cofre cerrado | Acciones: 3
Contraseña: ORBITA7392
No se pudo autenticar el archivo. Acciones: 2.

Contraseña: ORBITA-7392
Cofre abierto. REL-ESPECIAL incorporada al inventario.
Valor: 4 | Acciones restantes: 1
Recuerde entregarla en el campamento.
```

Estos intentos pertenecen al turno de Ana en la ronda tres. La apertura correcta incluye descifrado y recogida; no hay otra pantalla para introducir un código. El programa no muestra directamente la contraseña ensamblada antes de que el jugador la escriba.

En un templo explorado se muestra Explorado y vacío. Si el inventario está lleno, se informa Regrese al campamento para entregar antes de explorar. Las consultas no consumen acciones.

11 Jugada completa de las rondas uno a cuatro

EX-RED-01 usa el mapa del apartado 1. Ana y Luis salen de C3, en ese orden, con tres puntos por turno. La siguiente distribución de prueba revela los contenidos para explicar la jugada; el juego los descubre al explorar.

Casilla	Reliquia y valor	Pista
B3	REL-01, valor 3	1: ORBITA
C4	REL-02, valor 2	2: 7392
B1	REL-03, valor 2	Ninguna
A5	REL-04, valor 1	Ninguna
D2	REL-ESPECIAL, valor 4	Cofre

Las tablas cubren los dieciséis turnos. Fin indica posición; Puntos, total acumulado; Restan, presupuesto al cerrar. Las pistas reveladas en la ronda uno quedan disponibles para ambos.

Ronda	Jugador	Acciones en orden	Fin	Puntos	Restan
1	Ana	Mover a B3 → Explorar y recoger REL-01; pista 1 → Mover a C3	C3	0	0
1	Luis	Mover a C4 → Explorar y recoger REL-02; pista 2 → Mover a D4	D4	0	0
2	Ana	Entregar (+3 puntos) → Mover a D3 → Mover a D2	D2	3	0
2	Luis	Mover a C4 → Mover a C3 → Entregar (+2 puntos)	C3	2	0
3	Ana	Intentar ORBITA7392: falla → Abrir cofre y recoger REL-ESPECIAL → Mover a D3	D3	3	0
3	Luis	Mover a B3 → Mover a B2 → Mover a B1	B1	2	0
4	Ana	Mover a C3 → Entregar (+4 puntos) → Mover a C4	C4	7	0
4	Luis	Explorar y recoger REL-03 → Mover a B2 → Mover a B3	B3	2	0

Explicación de los resultados

En la primera ronda Ana recoge REL-01 y revela la pista 1; Luis recoge REL-02 y revela la pista 2. Los inventarios tienen objetos, pero la puntuación todavía es cero. Ambos entregan en la segunda ronda: Ana suma tres y Luis dos.

Ana llega al cofre con el inventario vacío. En la tercera ronda falla un intento, abre correctamente con el siguiente y se mueve a D3. Son tres acciones de costo uno. En la cuarta entrega la especial: queda con siete puntos. Luis recoge REL-03 y la transporta hacia C.

12 Jugada completa de las rondas cinco a ocho

Ronda	Jugador	Acciones en orden	Fin	Puntos	Restan
5	Ana	Mover a B4 → Mover a A4 → Mover a A5	A5	7	0
5	Luis	Mover a C3 → Entregar (+2 puntos) → Mover a D3	D3	4	0
6	Ana	Explorar y recoger REL-04 → Mover a A4 → Mover a B4	B4	7	0
6	Luis	Mover a D2 → Mover a E2 → Mover a E1	E1	4	0
7	Ana	Mover a C4 → Mover a C3 → Entregar (+1 puntos)	C3	8	0
7	Luis	Mover a D1 (costo 2) → Mover a C1	C1	4	0
8	Ana	Mover a D3 → Mover a D2 → Mover a C2	C2	8	0
8	Luis	Mover a B1 → Mover a B2 → Mover a B3	B3	4	0

Explicación de los resultados

En la quinta ronda Luis entrega REL-03 y alcanza cuatro puntos. Ana llega a A5, recoge REL-04 en la sexta y la entrega en la séptima: obtiene su octavo punto. La visita de Luis al cofre en la sexta encuentra ABIERTO y no produce otra reliquia.

En la séptima ronda Luis entra en D1, un pantano, por dos puntos y sale a C1 por uno. Solo ejecuta dos operaciones, pero usa su presupuesto completo. La octava ronda se juega normalmente; el agotamiento de las reliquias no finaliza antes el juego.

Jugador	Puntos	Reliquias entregadas	Inventario final	Resultado
Ana	8	REL-01, REL-ESPECIAL y REL-04	Vacío	Ganadora
Luis	4	REL-02 y REL-03	Vacío	Segundo

Los dieciséis turnos se verificaron contra el mapa, los costos y la capacidad. Las cinco reliquias fueron entregadas una sola vez y la suma final es doce. La comprobación criptográfica valida la contraseña correcta y rechaza una contraseña errónea o un ciphertext alterado.

13 Diseño orientado a objetos y requisitos técnicos

El proyecto utiliza Java (la misma versión instalada en el laboratorio) y un build reproducible con Maven. La API corre como proceso separado. Su URL base se carga desde configuración. Debe haber separación entre entidades, reglas, controlador, vista de consola y cliente HTTP.

- MVC obligatorio: el modelo no imprime ni solicita entradas; el controlador coordina y la vista presenta.
- Se aplicarán dos patrones a parte de MVC, excluyendo Singleton. Se recomienda C_____: cada acción tiene validación, costo y ejecución propios. FASE 2
- Una abstracción o interfaz con implementaciones polimórficas útiles. Por ejemplo, acciones de movimiento, exploración, entrega y cofre.
- Matriz para el tablero, listas para jugadores e inventarios, un conjunto para templos explorados y mapas para búsquedas por id. Justificar su uso.
- Encapsular cambios de estado y evitar que la vista modifique directamente inventarios, puntos o turnos.
- Cliente API y servicio criptográfico separados del modelo. Utilizar bibliotecas estándar para PBKDF2 y AES-GCM.
- Validar entradas y manejar errores de IO, HTTP y criptografía. Configurar tiempos máximos de red, sin reintentos infinitos. FASE 2
- Nombres claros, constantes para las reglas, enums para tipos y estados, y JavaDoc en clases y métodos públicos.

Responsabilidades sugeridas

Clase o interfaz	Responsabilidad
Partida	Ronda, turno, presupuesto, puntuación y finalización
Tablero y Casilla	Coordenadas, vecindad y terreno
Jugador y Participante	Registro del usuario y su estado en una partida
Reliquia e Inventario	Identidad, valor, capacidad y entrega
AccionJuego y variantes	Validación y ejecución polimórfica
ServicioCriptografico	Derivación y descifrado autenticado
ClienteApi y RepositorioPartidas	HTTP, JSON, carga y guardado
Controlador y VistaConsola	Casos de uso y entrada y salida

Las clases son sugerencias y no un número mínimo. La segunda fase sustituye la vista por JavaFX y adapta el controlador; mantiene las reglas y los DTO del contrato. No se solicita desarrollar interfaz gráfica en esta entrega.

14 Entregables y criterios de aceptación

- Código fuente Java, configuración de build y recursos. Debe compilar y ejecutarse fuera del IDE.
- README con requisitos, dirección de API, instrucciones de ejecución, reglas, guardado y errores conocidos.
- Informe PDF con integrantes, UML de clases, MVC, patrón adicional, estructuras y evidencia de los casos siguientes.
- Demostración de una partida generada, una carga a mitad de turno y la apertura del vector del anexo.
- Defensa individual. Fechas, medio de entrega y ponderaciones se completan por el docente.

ID	Caso mínimo de aceptación
CA01	Registrar y buscar; rechazar alias repetido y jugadores duplicados en una partida.
CA02	Crear tres partidas válidas; comprobar cantidades, restricciones y cambio de C, X y pistas para el mismo par.
CA03	Rechazar diagonal, fuera de mapa y presupuesto insuficiente sin alterar el estado; pantano cuesta dos.
CA04	Explorar y recoger en una acción; inventario lleno no revela ni retira; templo no repite su objeto.
CA05	Revelar dos pistas en templos diferentes y permitir a ambos jugadores consultarlas.
CA06	Abrir con contraseña correcta; fallo auténtico cuesta uno; datos corruptos de formato no cuestan.
CA07	Cofre requiere espacio y solo entrega una reliquia; entregar suma valores y vacía inventario.
CA08	Completar ocho rondas; comprobar reliquias sin entregar, desempate y victoria compartida.
CA09	Guardar a mitad de turno y cargar exactamente; simular consulta y guardado fallidos; reintentar sin duplicar efectos.
CA10	Repetir creación y guardado final: una partida y estadísticas sin duplicados; comprobar MVC y patrón.

Conviene construir primero el tablero y los turnos; luego inventario y puntuación; después API y guardado; al final integrar AES. Las pruebas deben comprobar reglas observables y fallos, no únicamente getters. El docente debe disponer de la API versión 2 antes de la evaluación.

Anexo A Glosario del juego y de programación

Término	Significado en este enunciado
Partida	Instancia con mapa, dos participantes y estado propios
Casilla y coordenada	Posición del tablero e identificación como C3
Vecina ortogonal	Casilla inmediata arriba, abajo, izquierda o derecha
Campamento	Inicio y lugar de entrega; símbolo C
Pantano	Terreno P cuya entrada cuesta dos puntos
Templo normal	Casilla T con una reliquia y, a veces, una pista
Cofre	Casilla X con una única reliquia cifrada
Explorar	Revelar un templo, recoger su reliquia y publicar su pista
Inventario	Reliquias transportadas; capacidad dos
Entrega	Retirar todas las reliquias en C y sumar su valor
Pista	Fragmento numerado de la contraseña, compartido al descubrirse
Turno y ronda	Intervención de un jugador y un turno de cada participante
Punto de acción	Unidad del presupuesto de tres por turno
Estado	Posiciones, objetos, puntos, pistas y progreso actual
POO y polimorfismo	Entidades con datos y métodos; variantes que ejecutan un mismo contrato
MVC	Modelo de reglas, vista de interacción y controlador que coordina
Command	Representa acciones mediante objetos con validación y ejecución
API y endpoint	Contrato de comunicación y operación identificada por método y ruta
Cliente y backend	Aplicación Java y servicio Python, respectivamente
JSON y DTO	Formato de datos y objeto que representa los datos transferidos
Snapshot	Copia completa del estado para guardarla
Persistencia	Conservación de datos entre ejecuciones
Revisión	Número que identifica una versión guardada
requestId	Identidad de una solicitud de creación para evitar duplicados
Idempotencia	Repetir la misma solicitud tiene un solo efecto

Término	Significado en este enunciado
Configuración inmutable	Mapa y asignaciones que no se regeneran al cargar
Semilla	Valor utilizado para reproducir el sorteo del mapa
JavaFX UML y JavaDoc	Interfaz gráfica, diagramas de diseño y documentación del código

Anexo B Vector de prueba del cofre

Los siguientes datos son una prueba pública reproducible de EX-RED-01. La contraseña se revela aquí para evaluar interoperabilidad. En partidas normales la API produce nuevos fragmentos, sal, nonce y ciphertext. El paquete no incluye contraseña ni clave derivada.

partidaId: EX-RED-01
cofreId: COFRE-01
algoritmo: AES/GCM/NoPadding
kdf: PBKDF2WithHmacSHA256
iteraciones: 100000
longitudClaveBits: 256
saltBase64: xBxRniX+mjbP3gI6qy93zA==
nonceBase64: BVmBDNDCL0Gw5CM6
tagBits: 128
aadBase64: RVgtUkVELTAxfENPRlJFLTAx
cifradoConTagBase64:
u/hosFuUuUt+RKGBQV9S/amTmkZd5SwCLtnzyn6/EgT9Y3Nmhs7G9JfRfXMZPce426aLkJRpuGdof1jG0
pikC+bkl1DmSNPrLlES17AuJ9UZaWvXl2pgbL84kWXKITU5noqdPQ==

Si una cadena Base64 ocupa varias líneas, concatenarlas sin espacios ni saltos. La etiqueta GCM ya está al final del campo cifradoConTagBase64; no se agrega otra. Todos los campos de longitud e iteraciones son números.

Resultado esperado

Contraseña: ORBITA-7392
AAD UTF-8: EX-RED-01|COFRE-01
Texto descifrado:
{"partidaId":"EX-RED-01","cofreId":"COFRE-01","reliquiaId":"REL-ESPECIAL","valor"
:4}

PBKDF2 deriva 32 bytes usando UTF-8 de la contraseña, HMAC-SHA256, la sal decodificada y 100000 iteraciones. AES-GCM recibe el nonce, la AAD y el ciphertext con etiqueta. El resultado autentica el archivo y confirma partidaId y cofreId.

Pruebas mínimas: contraseña correcta recupera el JSON; ORBITA7392 no autentica; alterar un byte del ciphertext no autentica. Se debe distinguir un resultado criptográfico fallido de un dato que no cumple el formato del contrato.

Anexo C Términos criptográficos y referencias

Término	Definición
AES	Cifrado simétrico que utiliza la misma clave para cifrar y descifrar
GCM	Modo de cifrado autenticado que comprueba la integridad de los datos
Contraseña	Cadena reconstruida a partir de las pistas 1 y 2
Clave y PBKDF2	Bytes para AES y función que los deriva desde contraseña, sal e iteraciones
HMAC-SHA256	Función utilizada dentro de PBKDF2
Sal	Bytes aleatorios públicos empleados en la derivación
Nonce o IV	Bytes usados por GCM; no se repiten al cifrar con la misma clave
Etiqueta o tag	Bytes que permiten autenticar ciphertext y AAD
AAD	Datos autenticados sin cifrar que vinculan el archivo con partida y cofre
Ciphertext y texto claro	Bytes cifrados y JSON legible recuperado
Base64 y UTF-8	Representación textual de bytes y codificación de texto, respectivamente
Vector de prueba	Datos completos con resultado conocido para comprobar una implementación

Base64 no cifra ni protege secretos. AES-GCM y PBKDF2 se utilizan mediante bibliotecas; el reto consiste en reunir pistas e integrar correctamente el descifrado. Los parámetros de esta actividad forman parte del contrato didáctico y no se presentan como una política de seguridad para otro sistema.

Referencias técnicas

Oracle Java 17 Cipher — https://docs.oracle.com/en/java/javase/17/docs/api/java.base/javax/crypto/Cipher.html

IETF RFC 8018 PBKDF2 — https://www.rfc-editor.org/rfc/rfc8018

NIST SP 800 38D GCM — https://csrc.nist.gov/pubs/sp/800/38/d/final

INFORMACION ADICIONAL

Para implementar la arquitectura de la API de Python cumpliendo con la separación del cliente API/servicio criptográfico y el modelo, y utilizando exclusivamente bibliotecas estándar (hashlib y cryptography / Crypto según convenga, o la librería estándar estándar de Python como hashlib.pbkdf2_hmac y cryptography.hazmat), la estructura óptima se divide en tres componentes independientes:

1. Servicio Criptográfico (crypto_service.py)

Encargado de derivar la clave mediante **PBKDF2-HMAC-SHA256** y empaquetar/descifrar el paquete con **AES-GCM-128/256**. Utiliza las bibliotecas hashlib y cryptography.

```
import base64
import json
import hashlib
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

class CryptoService:
    ITERATIONS = 100_000
    KEY_LENGTH = 32  # 256 bits

    @staticmethod
    def derive_key(password: str, salt_bytes: bytes) -> bytes:
        """Deriva una clave de 256 bits usando PBKDF2WithHmacSHA256."""
        return hashlib.pbkdf2_hmac(
            hash_name='sha256',
            password=password.encode('utf-8'),
            salt=salt_bytes,
            iterations=CryptoService.ITERATIONS,
            dklen=CryptoService.KEY_LENGTH
        )

    @staticmethod
    def encrypt_chest_payload(partida_id: str, cofre_id: str, reliquia_id: str, valor: int, password: str,
salt_bytes: bytes, nonce_bytes: bytes) -> dict:
        """Genera el paquete cifrado en formato Base64 para ser consumido por la API/Java."""
        key = CryptoService.derive_key(password, salt_bytes)

        # Payload JSON UTF-8
        payload = json.dumps({
            "partidaId": partida_id,
            "cofreId": cofre_id,
            "reliquiaId": reliquia_id,
            "valor": valor
        }).encode('utf-8')

        # AAD: partidaId|cofreId
        aad = f"{partida_id}|{cofre_id}".encode('utf-8')

        # AES-GCM (añade la etiqueta de 16 bytes al final del ciphertext)
        aesgcm = AESGCM(key)
```

```
    ciphertext_with_tag = aesgcm.encrypt(nonce_bytes, payload, aad)

    return {
      "partidaId": partida_id,
      "cofreId": cofre_id,
      "algoritmo": "AES/GCM/NoPadding",
      "kdf": "PBKDF2WithHmacSHA256",
      "iteraciones": CryptoService.ITERATIONS,
      "longitudClaveBits": CryptoService.KEY_LENGTH * 8,
      "saltBase64": base64.b64encode(salt_bytes).decode('utf-8'),
      "nonceBase64": base64.b64encode(nonce_bytes).decode('utf-8'),
      "tagBits": 128,
      "aadBase64": base64.b64encode(aad).decode('utf-8'),
      "cifradoConTagBase64": base64.b64encode(ciphertext_with_tag).decode('utf-8')
    }
```

2. Modelo de Dominio (`models.py`)

Mantiene la lógica pura del juego y la definición de las entidades (tablero, cofres, partidas) sin importar clientes HTTP ni librerías criptográficas directas.

```
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class Casilla:
  coordenada: str  # Ej: "D2"
  tipo: str       # 'C', 'S', 'P', 'T', 'X'

@dataclass
class Cofre:
  cofre_id: str
  posicion: str
  paquete_cifrado: dict  # Almacena el payload generado previamente por CryptoService
  abierto: bool = False

@dataclass
class PartidaConfig:
  partida_id: str
  jugadores: List[str]
  tablero: List[Casilla]
  cofre: Cofre
```

pistas: List[dict]

3. Cliente API / Endpoints HTTP (`api_client.py` / `main.py`)
Se encarga únicamente de recibir las peticiones REST (usando frameworks estándar de Python como **FastAPI** o **Flask**), consultar el modelo o el servicio de persistencia y retornar las respuestas JSON.

```
from fastapi import FastAPI, HTTPException
import os

app = FastAPI(title="API Expedición Isla de las Reliquias", version="1.0")

# Repositorio/Estado en memoria o persistencia JSON
PARTIDAS_DB = {}

@app.get("/api/v1/salud")
def consultar_salud():
   return {"estado": "Disponible", "versionContrato": 1}

@app.get("/api/v1/partidas/{partida_id}/cofre")
def obtener_cofre(partida_id: str):
   """Devuelve el paquete cifrado completo sin exponer contraseñas ni claves."""
   if partida_id not in PARTIDAS_DB:
      raise HTTPException(status_code=404, detail={"codigo": "NOT_FOUND", "mensaje": "Partida no
encontrada"})

   partida = PARTIDAS_DB[partida_id]
   return partida.cofre.paquete_cifrado
```

Puntos clave de la separación:

- **Aislamiento de responsabilidades:** El modelo (models.py) no conoce los detalles de implementación de CryptoService ni de las rutas HTTP (api_client.py).
- **Cumplimiento del contrato API:** El paquete devuelto por la API en /partidas/{id}/cofre contiene unívocamente saltBase64, nonceBase64, aadBase64 y cifradoConTagBase64 sin revelar jamás la clave AES ni el texto en claro.